

Tensor Foundation

Security Assessment

September 27th, 2024 — Prepared by OtterSec

Ajay Shankar Kunapareddy d1r3wolf@osec.io

Tamta Topuria tamta@osec.io

Robert Chen r@osec.io

Table of Contents

Executive Summary **3**

Overview 3

Key Findings 3

Scope **4**

Findings **5**

Vulnerabilities **6**

OS-TFN-ADV-00 | Flawed Logic in case of Shared Escrow 9

OS-TFN-ADV-01 | Ineffective Shared Escrow Validation 11

OS-TFN-ADV-02 | Missing Account Authenticity Checks 12

OS-TFN-ADV-03 | Improper Rent Calculation 13

OS-TFN-ADV-04 | Legacy NFT Validation Gaps 14

OS-TFN-ADV-05 | Absence of Royalty Enforcement 15

OS-TFN-ADV-06 | Utilization of Improper Payer Account 16

OS-TFN-ADV-07 | Denial of Service Due to ATA Ownership Change 18

OS-TFN-ADV-08 | Spurious Abort on Excess Balance Amount 20

OS-TFN-ADV-09 | Unsafe Account Closure Due to Missing Updates 21

OS-TFN-ADV-10 | Whitelist Version Compatibility Conflict 22

OS-TFN-ADV-11 | Missing Timestamp Update 23

OS-TFN-ADV-12 | Inconsistent Validation of Starting Price 24

General Findings **25**

OS-TFN-SUG-00 | Incorrect Rent Transfer 26

OS-TFN-SUG-01 | Missing Validation Logic 27

© 2024 Otter Audits LLC. All Rights Reserved.

OS-TFN-SUG-02 | Redundant/Unutilized Code 28

OS-TFN-SUG-03 | Code Maturity 30

OS-TFN-SUG-04 | Code Refactoring 32

OS-TFN-SUG-05 | Code Clarity 34

OS-TFN-SUG-06 | Code Optimizations 35

Appendices

Vulnerability Rating Scale **36**

Procedure **37**

© 2024 Otter Audits LLC. All Rights Reserved.

01 — Executive Summary

Overview

Tensor engaged OtterSec to assess the `pricelock`, `amm`, `marketplace`, `whitelist`, `escrow`, `fees`, and `toolbox` programs. This assessment was conducted between August 16th and September 18th, 2024. For more information on our auditing methodology, refer to Appendix B.

Key Findings

We produced 20 findings throughout this audit engagement.

In particular, we identified a vulnerability concerning the lack of state variable updates, resulting in the closure of margin accounts with active bids (OS-TFN-ADV-09), and another issue where the rent is calculated based on an account's lamports instead of the rent-exemption reserve, potentially disrupting trades and the transfer of excess SOL to the order vault account upon account closure (OS-TFN-ADV-03). It is possible for the maker to change the owner of their associated token account (ATA), preventing the taker from claiming their collateral (OS-TFN-ADV-07).

Furthermore, we highlighted several inconsistencies in the implementation of the shared escrow logic (OS-TFN-ADV-00,OS-TFN-ADV-01), and the lack of proper seed and initialization checks for the metadata and edition accounts, rendering the system vulnerable to manipulation and failure to verify the authenticity of the NFT mint (OS-TFN-ADV-03).

We also made recommendations around modifications to the codebase for improved efficiency (OS-TFN-SUG-04) and suggested the need to ensure adherence to coding best practices (OS-TFN-SUG-03). We further highlighted several instances where proper validation was not done and advised incorporating those checks into the codebase (OS-TFN-SUG-01) and also to streamline the code to further increase efficiency and reduce complexity (OS-TFN-SUG-06). Additionally, we recommended removing certain instances of unutilized code for improved readability and maintainability (OS-TFN-SUG-02).

© 2024 Otter Audits LLC. All Rights Reserved.

02 — Scope

The source code was delivered to us in a Git repository at https://github.com/tensor-foundation. This audit was performed against commits fcf75d3, 961670d, 5191182, 22686eb, ca3e7d9, c7cb5bf, and 8daebf9 corresponding to `pricelock`, `amm`, `marketplace`, `whitelist`, `escrow`, `fees`, and `toobox` programs respectively. At the time of this report, follow-up reviews are ongoing against commits 35aa16c, b977032, 4f6daae, 8850a44, and eccc9f1 corresponding to `amm`, `marketplace`, `whitelist`, `escrow`, and `fees`, and programs respectively.

A brief description of the programs is as follows:

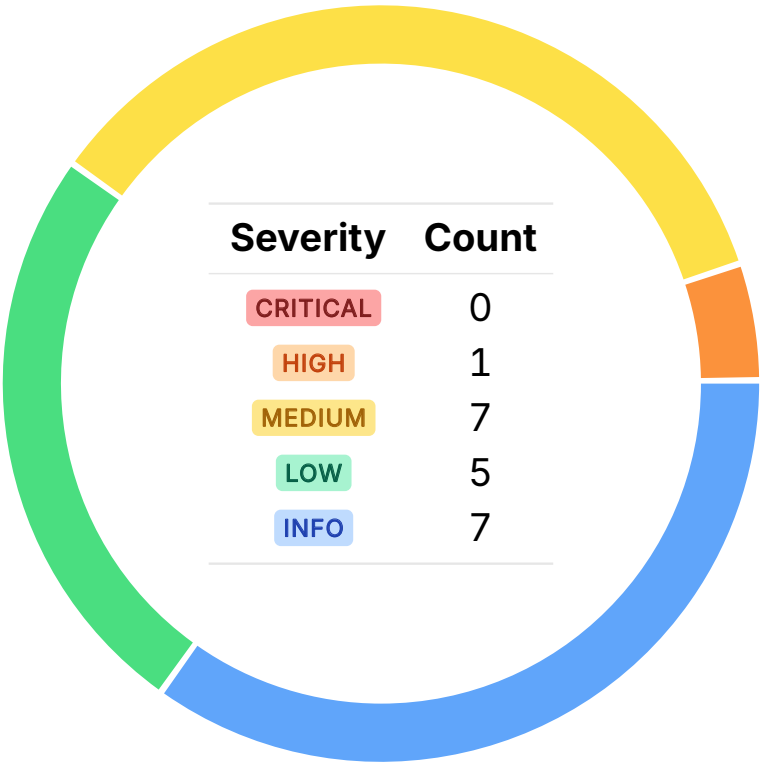
Name	Description
price-lock	Enables locking a buy or sell price on NFTs.
amm	Implementation of the second version of the Tensor AMM program.
marketplace	Implementation of an NFT marketplace.
whitelist	The whitelist program enables the on-chain verification of a collection.
escrow	An implementation of a shared escrow for all Tensor protocols.
fees	Fees PDA and collection program.
toolbox	Contains utility libraries for Solana development.

© 2024 Otter Audits LLC. All Rights Reserved.

03 — Findings

Overall, we reported 20 findings.

We split the findings into **vulnerabilities** and **general findings**. Vulnerabilities have an immediate impact and should be remediated as soon as possible. General findings do not have an immediate impact but will aid in mitigating future vulnerabilities.



Severity	Count
CRITICAL	0
HIGH	1
MEDIUM	7
LOW	5
INFO	7

© 2024 Otter Audits LLC. All Rights Reserved.

04 — Vulnerabilities

Here, we present a technical analysis of the vulnerabilities we identified during our audit. These vulnerabilities have *immediate* security implications, and we recommend remediation as soon as possible.

Rating criteria can be found in Appendix A.

ID	Severity	Status	Description
OS-TFN-ADV-00	HIGH	RESOLVED	`try_autoclose_pool` incorrectly assumes `pool.amount` reflects available funds when `shared_escrow` is configured, resulting in invalid verification.
OS-TFN-ADV-01	MEDIUM	RESOLVED	The shared escrow validation in `create_pool` utilizes an uninitialized `pool.config.pool_type`, rendering it ineffective.
OS-TFN-ADV-02	MEDIUM	RESOLVED	There is a lack of proper seed and initialization checks for the `metadata`, `edition`, and `master_edition` accounts, rendering the system vulnerable to manipulation and failure to verify the authenticity of the NFT mint.
OS-TFN-ADV-03	MEDIUM	RESOLVED	The rent is calculated based on an account's lamports instead of the rent-exemption reserve, potentially disrupting trades and the transfer of excess `SOL` to the order vault account upon account closure.
OS-TFN-ADV-04	MEDIUM	RESOLVED	The legacy standard lacks proper validation for the `master_edition` account in non-pNFTs, risking fungibility, and omits seed checks for the `metadata` account in `TakeBidLegacy`.

© 2024 Otter Audits LLC. All Rights Reserved.

OS-TFN-ADV-05	MEDIUM	RESOLVED	`process_buy_legacy_spl` utilizes `optional_royalty_pct` without verifying if royalties are enforced on `metadata.token_standard`, potentially bypassing intended royalty enforcement.
OS-TFN-ADV-06	MEDIUM	RESOLVED	`process_close_expired_listing_core` and `process_delist_core` incorrectly utilize the `list_state` and `rent_destination` accounts respectively as the payer for the transfer CPI.
OS-TFN-ADV-07	MEDIUM	RESOLVED	It is possible for the maker to change the owner of their associated token account (ATA), preventing the taker from claiming their collateral.
OS-TFN-ADV-08	LOW	RESOLVED	`verify_empty_balance` may abort unexpectedly in `take_bid_shared` if external lamports are transferred into the `bid_state` account, as it strictly checks for an exact balance match with rent.
OS-TFN-ADV-09	LOW	RESOLVED	The `pools_attached` field is not updated, resulting in a scenario where a margin account containing active bids may be closed.
OS-TFN-ADV-10	LOW	RESOLVED	`realloc_whitelist` restricts updates to whitelists with version 1, but `init_update_whitelist` may upgrade versions, creating a conflict where updated whitelists are ineligible for reallocation.
OS-TFN-ADV-11	LOW	RESOLVED	The `updated_at` field in `BidState` is not updated in `take_bid_shared`.

© 2024 Otter Audits LLC. All Rights Reserved.

OS-TFN-ADV-12	LOW	RESOLVED	The starting price validation for pools is currently applied only in the `create_pool` instruction and not in `edit_pool`.

© 2024 Otter Audits LLC. All Rights Reserved.

Flawed Logic in case of Shared Escrow HIGH OS-TFN-ADV-00

Description

The vulnerability in `pool::try_autoclose_pool` lies in the logic for handling `PoolType::Token`. Specifically, the function checks the `pool.amount` field to decide whether the pool has enough SOL to purchase another NFT, but it does not account for the case where a `shared_escrow` is configured. The logic assumes if `pool.currency == Pubkey::default`, it indicates the pool utilizes SOL (not a token). `pool.amount` represents the SOL available in the pool. The pool should be closed if `pool.amount` is less than the price needed to purchase another NFT.

>_ *amm/program/src/state/pool.rs* RUST

```rust
pub fn try_autoclose_pool<'info>(
    pool: &Account<'info, Pool>,
    rent_payer: AccountInfo<'info>,
    owner: AccountInfo<'info>,
) -> Result<()> {
    match pool.config.pool_type {
        PoolType::Trade => (), // Cannot be auto-closed
        PoolType::Token => {
            // Not enough SOL to purchase another NFT, so we can close the pool.
            if pool.currency == Pubkey::default()
                && pool.amount < pool.current_price(TakerSide::Sell)?
            {
                close_pool(pool, rent_payer, owner)?;
            }
        }
        [...]
    }

    Ok(())
}
```

However, when a `shared_escrow` is configured, the individual `pool.amount` field will always be zero because the funds are not stored in the pool itself. Instead, the funds are managed in the `shared_escrow` account. This causes the condition `pool.amount < pool.current_price(TakerSide::Sell)?` to always evaluate to true, even if sufficient funds exist in the `shared_escrow`. As a result, the pool may be erroneously closed while it is still active and has enough funds to continue operations.

Remediation

Check if `shared_escrow == pubkey::default` and in that case utilize the `pool.amount` directly.

© 2024 Otter Audits LLC. All Rights Reserved.

Patch

Resolved in #94.

© 2024 Otter Audits LLC. All Rights Reserved.

Ineffective Shared Escrow Validation MEDIUM OS-TFN-ADV-01

Description

In `CreatePool`, the `shared_escrow` field is constrained with the following check: `pool.config.pool_type != NFT`. This constraint is intended to ensure that a pool using shared escrow cannot have a `PoolType::NFT` configuration. However, the pool account is not yet initialized when this constraint is checked and `pool.config.pool_type` does not contain any meaningful data. Thus, any constraint referencing `pool.config.pool_type` before initialization is ineffective.

>_ *amm/program/src/instructions/create_pool.rs* RUST

```rust
/// Instruction accounts.
#[derive(Accounts)]
#[instruction(args: CreatePoolArgs)]
pub struct CreatePool<'info> {
    [...]
    #[account(
        has_one = owner @ ErrorCode::WrongOwner,
        constraint = pool.config.pool_type != PoolType::NFT @ ErrorCode::CannotUseSharedEscrow,
    )]
    pub shared_escrow: Option<Account<'info, MarginAccount>>,
    [...]
}
```

Remediation

Apply this constraint to the incoming arguments (`args.config.pool_type`) instead of the uninitialized `pool.config.pool_type`.

Patch

Resolved in 87ad4a7.

© 2024 Otter Audits LLC. All Rights Reserved.

Missing Account Authenticity Checks MEDIUM OS-TFN-ADV-02

Description

In `shared_accounts` within the `MplxShared` structure, the `metadata` and `edition` accounts are essential to validate an NFT's properties, such as its uniqueness and associated data. The vulnerability concerns the lack of seed checks to verify the existence of the `metadata` and `edition` for the provided mint.

>_ *amm/program/src/instructions/shared_accounts.rs* RUST

```rust
pub struct MplxShared<'info> {
    [...]
    /// The Token Metadata metadata account of the NFT.
    /// CHECK: ownership, structure and mint are checked in assert_decode_metadata.
    #[account(mut)]
    pub metadata: UncheckedAccount<'info>,

    // ------------------------------------ pNft
    /// The Token Metadata edition account of the NFT.
    /// CHECK: seeds checked on Token Metadata CPI
    //note that MASTER EDITION and EDITION share the same seeds, and so it's valid to check them
        ↪ here
    pub edition: UncheckedAccount<'info>,
    [...]
}
```

Additionally, in the transfer functionality, only the `metadata` is verified, and the `master_edition` is not utilized for the non-pNFT case. Thus, the `master_edition` account is not utilized or checked in the non-pNFT case. This is crucial because without verifying the existence of the master edition, there is no confirmation that the mint is truly non-fungible.

Remediation

implement PDA (Program Derived Address) and initialization checks to ensure that the `metadata`, `edition`, and `master_edition` accounts are properly validated.

Patch

Resolved in #94.

© 2024 Otter Audits LLC. All Rights Reserved.

Improper Rent Calculation MEDIUM OS-TFN-ADV-03

Description

`price-lock` utilizes `token_account.get_lamports` to determine the rent instead of calculating the proper rent-exemption reserve. If an attacker (`maker`) deposits a large amount of `SOL` (more than the rent-exemption reserve) into an account, it may prevent trades from executing. Since the program only expects a minimal amount for rent and does not handle large balances, the excess `SOL` will block the processing of the trade, rendering the option worthless.

For example, the `TAmmSellNftTokenPool`, `get_lamports` is utilized to determine the amount of `SOL` held in `order_ta` account. If a very large amount of `SOL` is deposited into the `order_ta` account, the program will assume this large balance represents the necessary rent to perform operations. The program will fail to execute trades because the logic incorrectly assumes that the large `SOL` deposit represents the required rent.

>_ *instructions/tamm/tamm_sell_nft_token_pool.rs* RUST

```rust
fn return_rent(&self, order_seeds: [&[&[u8]]; 1]) -> Result<()> {
    let token_account_rent = self.token.order_ta.get_lamports();
    let record_rent = Rent::get()?.minimum_balance(TokenRecord::LEN);
    // For the token pool, the seller receives everything back.
    let excess_rent = 2 * token_account_rent + record_rent;
    self.trade.return_rent_for_sell(excess_rent, order_seeds)
}
```

The `return_rent` logic tries to return any excess rent from `order_ta` back to the `order_vault`. It assumes that all `SOL` in the `order_ta` account should be returned. Since the program does not calculate the actual rent exemption correctly, it transfers a large portion of `SOL`, which includes both the genuine rent and the attacker's deposit, into the `order_vault`, resulting in improper `SOL` transfers and an imbalance in the vault's holdings. Thus, the option becomes worthless, as no trades may be executed, locking up trade executions.

Remediation

Ensure that the rent exemption reserve is properly calculated utilizing the size of the account instead of its lamports.

Patch

Resolved in 84d4928.

© 2024 Otter Audits LLC. All Rights Reserved.

Legacy NFT Validation Gaps MEDIUM OS-TFN-ADV-04

Description

The vulnerability relates to verification gaps in `TakeBidLegacy` and the handling of legacy standards for NFTs (non-programmable NFTs). The `master_edition` account ensures the non-fungibility of an NFT. However, the PDA validation (Program-Derived Address) for `master_edition` is missing for non-pNFTs. Additionally, the `metadata` account's seed checks are missing in the `TakeBidLegacy` context.

Remediation

Add PDA, initialized checks on `master_edition` in all legacy standard instructions and metadata account checks in `TakeBidLegacy`.

Patch

Resolved in #62.

© 2024 Otter Audits LLC. All Rights Reserved.

Absence of Royalty Enforcement MEDIUM OS-TFN-ADV-05

Description

In `buy_spl::process_buy_legacy_spl` the `optional_royalty_pct` parameter is currently utilized without verifying whether the royalties are actually enforced based on the `metadata.token_standard`. `optional_royalty_pct` is a user-provided parameter specifying the percentage of royalties to be paid to the creators. `metadata.token_standard` indicates the standard of the token. Some token standards enforce royalties at a protocol level. By directly utilizing `optional_royalty_pct`, the token standard that requires a specific amount of royalties may receive the incorrect amount if the user-supplied amount is inappropriate.

>_ *marketplace/program/src/instructions/legacy/buy_spl.rs* RUST

```rust
pub fn process_buy_legacy_spl<'info, 'b>(
    ctx: Context<'_, 'b, '_, 'info, BuyLegacySpl<'info>>,
    max_amount: u64,
    optional_royalty_pct: Option<u16>,
    authorization_data: Option<AuthorizationDataLocal>,
) -> Result<()> {
    [...]
      let creator_fee = calc_creators_fee(
        metadata.seller_fee_basis_points,
        amount,
        optional_royalty_pct,
    )?;
    [...]
}
```

Remediation

Ensure that if royalty is enforced for a certain token standard, the system overrides any user-provided `optional_royalty_pct` and applies a predefined percentage.

Patch

Resolved in #58.

© 2024 Otter Audits LLC. All Rights Reserved.

Utilization of Improper Payer Account MEDIUM OS-TFN-ADV-06

Description

In `close_expired_listing::process_close_expired_listing_core`, the `list_state` account is currently utilized as the payer for the `TransferV1CpiBuilder` CPI (Cross-Program Invocation). The `list_state` account is a program-owned account that holds metadata and operational data for the listing. Such accounts are generally funded only with enough lamports to cover rent. If the `list_state` account lacks sufficient lamports to cover the cost of the CPI invocation, the transaction will fail, resulting in the cleanup of the expired listing to revert.

>_ *marketplace/program/src/instructions/mpl_core/close_expired_listing.rs* RUST

```rust
pub fn process_close_expired_listing_core<'info>(
    ctx: Context<'_, '_, '_, 'info, CloseExpiredListingCore<'info>>,
) -> Result<()> {
    [...]
    TransferV1CpiBuilder::new(&ctx.accounts.mpl_core_program)
        .asset(&ctx.accounts.asset)
        .authority(Some(&ctx.accounts.list_state.to_account_info()))
        .new_owner(&ctx.accounts.owner.to_account_info())
        .payer(&ctx.accounts.list_state.to_account_info()) // pay for what?
        .collection(ctx.accounts.collection.as_ref().map(|c| c.as_ref()))
        .invoke_signed(&[&ctx.accounts.list_state.seeds()])?;
    [...]
}
```

Similarly, in `delist::process_delist_core` the `rent_destination` account is utilized as the payer in the `TransferV1CpiBuilder`. This enforces an unnecessary requirement that the `rent_destination` account should be a signer. The `rent_destination` account is not supposed to be responsible for paying transaction fees. Its role is to receive any refunded rent when the `list_state` account is closed, not to pay for the transfer itself. As a result of the constraint, the delisting may fail if the `rent_destination` is not a signer.

>_ *marketplace/program/src/instructions/mpl_core/delist.rs* RUST

```rust
pub fn process_delist_core<'info>(
    ctx: Context<'_, '_, '_, 'info, DelistCore<'info>>,
) -> Result<()> {
    [...]
  TransferV1CpiBuilder::new(&ctx.accounts.mpl_core_program)
        .asset(&ctx.accounts.asset)
        .authority(Some(&ctx.accounts.list_state.to_account_info()))
        .new_owner(&ctx.accounts.owner.to_account_info())
        .payer(&ctx.accounts.rent_destination) // pay for what?
```

© 2024 Otter Audits LLC. All Rights Reserved.

```
            .collection(ctx.accounts.collection.as_ref().map(|c| c.as_ref()))
            .invoke_signed(&[&ctx.accounts.list_state.seeds()])?;
        [...]
    }
```

Remediation

Replace `list_state` with a user-provided payer account in `process_close_expired_listing_core`, and utilize the `DelistCore.owner` account in `process_delist_core`.

Patch

Resolved in #62.

© 2024 Otter Audits LLC. All Rights Reserved.

Denial of Service Due to ATA Ownership Change MEDIUM OS-TFN-ADV-07

Description

As a result of the `WithdrawCollateral` token account checks in `WithdrawCollateralShared` within `validate`, there is a possibility that a Denial of Service (DoS) attack may occur if the maker (the original creator of the lock) intentionally changes their Associated Token Account (ATA) ownership to prevent the taker (the buyer) from claiming their collateral.

>_ *program/src/state.rs* RUST

```rust
pub fn validate(&mut self, collateral_type: CollateralType, to_maker: bool) -> Result<()> {
    [...]
    // If returning to maker: cannot
    if to_maker {
        require!(
            !order_state.collateral_returned,
            TLockError::CollateralAlreadyReturned
        );
    } else {
        require!(
            order_state.collateral_returned,
            TLockError::CollateralNotReturned
        );
    }

    if order_state.is_lock_closeable_by_anyone()? {
        // NB: anyone can permissionlessly return the collateral back to the maker.
    } else {
        require!(
            order_state.taker.unwrap() == self.signer.key(),
            TLockError::InvalidSigner
        );
    }
    [...]
    Ok(())
}
```

Proof of Concept

1. The maker creates an `NFT` lock and deposits their `NFT` into the lock, which is held as collateral.
2. The taker buys the locked `NFT` from the maker, sending yield to the maker. As a result, the value of the `NFT` increases, which implies that the taker stands to benefit if they sell it.
3. The taker sells the `NFT` at a higher price. As a result, the price of the `NFT` drops, implying it is possible to buy back the `NFT` at a lower price.

© 2024 Otter Audits LLC. All Rights Reserved.

4. The taker buys the `NFT` again at a lower price. Consequently, the `NFT`'s value drops drastically.
5. At this point, the maker changes the owner of their `ATA` (which holds the `SOL` collateral).
6. If the taker, now in possession of the `NFT` again, wishes to return the `NFT` to the maker and withdraw the `SOL` collateral, they will be unable to do so due to the change in the maker's `ATA` ownership.

Remediation

Create the ATA only if it does not already exist.

Patch

Resolved in d8df474.

© 2024 Otter Audits LLC. All Rights Reserved.

Spurious Abort on Excess Balance Amount LOW OS-TFN-ADV-08

Description

`BidState::verify_empty_balance` in `take_bid_shared` may fail spuriously if an external entity transfers lamports into the `bid_state` account. This will result in the balance exceeding the expected rent balance. `verify_empty_balance` checks whether the lamports in the account are exactly equal to the rent-exempt balance (`bid_rent`). If there are more lamports, this check will fail, resulting in the function aborting with the `TcompError::BidBalanceNotEmptied` error.

>_ *program/src/state.rs* RUST

```rust
pub fn verify_empty_balance(bid_state: &Account<BidState>) -> Result<()> {
    let info: &AccountInfo = bid_state.as_ref();
    let bid_rent = Rent::get()?.minimum_balance(info.data_len());
    require!(
        info.try_lamports()? == bid_rent,
        TcompError::BidBalanceNotEmptied
    );
    Ok(())
}
```

Remediation

Modify `verify_empty_balance` to ignore any excess lamports and allow closing the account as long as the balance is greater than or equal to the rent-exempt minimum.

Patch

Resolved in 442a514.

© 2024 Otter Audits LLC. All Rights Reserved.

Unsafe Account Closure Due to Missing Updates LOW OS-TFN-ADV-09

Description

Currently in `Escrow`, the majority of the fields in `TSwap` and `MarginAccount` structures are not utilized. Specifically, the `pools_attached` field in the `MarginAccount` structure is intended to track how many liquidity pools or active bids are attached to a margin account. This field is supposed to prevent a user from closing a margin account if it has active positions.

>_ instructions/close_margin_account.rs RUST

```rust
impl<'info> Validate<'info> for CloseMarginAccount<'info> {
    fn validate(&self) -> Result<()> {
        if self.margin_account.pools_attached > 0 {
            throw_err!(ErrorCode::MarginInUse);
        }
        Ok(())
    }
}
```

However, because the code no longer updates this field, it may be out of sync with the actual number of open positions corresponding to a margin account. Thus, the system will falsely believe there are no active pools attached to the margin account, successfully passing the check in the `CloseMarginAccount` instruction, allowing for unsafe account closure.

Remediation

Ensure the `pools_attached` field is properly updated whenever a new bid is opened.

Patch

Tensor has acknowledged the vulnerability and accepted risk.

© 2024 Otter Audits LLC. All Rights Reserved.

Whitelist Version Compatibility Conflict LOW OS-TFN-ADV-10

Description

There is an inconsistency between the logic of the `realloc_whitelist` and `init_update_whitelist` modules, specifically concerning how the whitelist version is managed and validated. `realloc_whitelist::validate` explicitly checks that the whitelist account's version is exactly 1.

>_ *whitelist/program/src/instructions/realloc_whitelist.rs* RUST

```rust
impl<'info> Validate<'info> for ReallocWhitelist<'info> {
    fn validate(&self) -> Result<()> {
        //can only migrate those with old version, intentionally keeping hardcorded to avoid
            ↪ errors
        if self.whitelist.version != 1 {
            throw_err!(ErrorCode::BadWhitelist);
        }
        Ok(())
    }
}
```

However, `init_update_whitelist`, which is responsible for initializing or updating a Whitelist account, assigns the value of `CURRENT_WHITELIST_VERSION` to the `whitelist.version` field. If `CURRENT_WHITELIST_VERSIO` is incremented to 2, the updated `whitelist` will be incompatible with `realloc_whitelist`, effectively blocking any reallocation for that `whitelist`.

Remediation

Check the `whitelist` version before updating in `realloc_whitelist`.

Patch

Tensor has acknowledged the vulnerability and accepted risk.

© 2024 Otter Audits LLC. All Rights Reserved.

Missing Timestamp Update LOW OS-TFN-ADV-11

Description

The `updated_at` field in the `BidState` structure is not updated during the execution of `take_bid_shared` and stakeholders will not have an accurate record of when the bid was last modified.

Remediation

Ensure `updated_at` field is updated whenever any modification is made to the `BidState`.

Patch

Resolved in #62.

© 2024 Otter Audits LLC. All Rights Reserved.

Inconsistent Validation of Starting Price LOW OS-TFN-ADV-12

Description

In the current code, there is a check for the `starting_price` field in `create_pool::process_create_pool`. This check ensures that the starting price of the pool is not less than 1 when a new pool is created. However, this validation is missing in the `edit_pool` instruction in `validate_pool_config`. The `edit_pool` instruction allows an existing pool to be modified, and the `starting_price` field may be updated. If the `starting_price` is not validated in `edit_pool`, an invalid value may be set for an existing pool.

>_ *amm/program/src/instructions/edit_pool.rs* RUST

```rust
fn validate_pool_config(&self, new_config: Option<PoolConfig>) -> Result<()> {
    let new_config = match new_config {
        Some(config) => config,
        None => return Ok(()),
    };
    //cannot change pool type
    if self.pool.config.pool_type != new_config.pool_type {
        throw_err!(ErrorCode::WrongPoolType);
    }
    new_config.validate()
}
```

Remediation

Move the validation logic for `starting_price` into `PoolConfig::validate`. This will ensure it is applied consistently in both the `create_pool` and `edit_pool` instructions. This way, the validation is centralized and reusable, which improves code maintainability.

Patch

Resolved in #96.

© 2024 Otter Audits LLC. All Rights Reserved.

05 — General Findings

Here, we present a discussion of general findings during our audit. While these findings do not present an immediate security impact, they represent anti-patterns and may result in security issues in the future.

ID	Description
OS-TFN-SUG-00	`CloseOrder` transfers rent from the source token account to the taker in all cases, even when the account is funded by the maker.
OS-TFN-SUG-01	There are several instances where proper validation is not performed, resulting in potential security issues.
OS-TFN-SUG-02	The protocol contains multiple cases of irrelevant or redundant code, which may be removed.
OS-TFN-SUG-03	Suggestions to ensure adherence to coding best practices.
OS-TFN-SUG-04	Recommendations for modifying the codebase to mitigate potential security issues and improve functionality.
OS-TFN-SUG-05	Modifications to better code readability and clarity.
OS-TFN-SUG-06	The overall code may be streamlined further to increase efficiency and reduce complexity.

© 2024 Otter Audits LLC. All Rights Reserved.

Incorrect Rent Transfer

OS-TFN-SUG-00

Description

The vulnerability in `CloseOrder` arises from a logic flaw in how rent is handled during the closure of the `order_vault` account, specifically the `source_ta` account, which is the token account associated with the order vault. When the maker creates the order, they fund the `order_vault`, which includes creating and funding the `source_ta` account. This account holds some amount of `SOL` (rent-exempt reserve). However, when the order is closed, the rent from the `source_ta` account is always transferred to the taker, even when the `source_ta` was funded by the maker.

>_ instructions/tamm/tamm_sell_nft_token_pool.rs RUST

```rust
fn return_rent(&self, order_seeds: [&[&[u8]]; 1]) -> Result<()> {
    let token_account_rent = self.token.order_ta.get_lamports();
    let record_rent = Rent::get()?.minimum_balance(TokenRecord::LEN);
    // For the token pool, the seller receives everything back.
    let excess_rent = 2 * token_account_rent + record_rent;
    self.trade.return_rent_for_sell(excess_rent, order_seeds)
}
```

This results in an unfair transfer of `SOL` to the taker, even though it was the maker who originally paid for the rent-exempt reserve of the `source_ta`. In essence, the taker is benefiting from the maker's `SOL` without having contributed to it.

Remediation

Update the rent transfer logic to ensure that the rent-exempt reserve from the `source_ta` account is returned to the correct party.

© 2024 Otter Audits LLC. All Rights Reserved.

Missing Validation Logic OS-TFN-SUG-01

Description

1. `transfer_cnft` relies heavily on the correctness of parameters such as `root`, `nonce`, `index`, `data_hash`, and `creator_hash`. Both `CloseExpiredListing` and `Delist` instructions pass these parameters without additional validation, which may result in incorrect `NFT` transfers if the parameters are mismatched. Add additional validation steps to verify the correctness of these parameters.
2. Add a pause functionality to effectively halt all activity in the pool before making any edits, preventing any further interaction with the pool until the owner is done editing.

>_ program/src/instructions/take_bid_common.rs RUST

```rust
pub fn process_edit_pool(ctx: Context<EditPool>, args: EditPoolArgs) -> Result<()> {
    let pool = &mut ctx.accounts.pool;
    if let Some(new_config) = args.new_config {
        pool.config = new_config;
    }
    if let Some(cosigner) = args.cosigner {
        pool.cosigner = NullableOption::new(cosigner);
    }
    if let Some(max_taker_sell_count) = args.max_taker_sell_count {
        pool.valid_max_sell_count(max_taker_sell_count)?;
        pool.max_taker_sell_count = max_taker_sell_count;
    }
    if args.reset_price_offset {
        pool.price_offset = 0;
    }
    let timestamp = Clock::get()?.unix_timestamp;
    [...]
}
```

Remediation

Incorporate the above validations into the codebase.

Patch

1. Resolved in b95be94.

© 2024 Otter Audits LLC. All Rights Reserved.

Redundant/Unutilized Code

OS-TFN-SUG-02

Description

1. In the bidding process `bid_state.currency` is currently unutilized.
2. `shard_num` in `toolbox::common` has two conditions, but it will only ever match the first one. Thus, the second condition may be removed.
3. In `bid::process_bid`, utilize `assert_expiry` to eliminate code redundancy.

>_ program/src/state/pool.rs RUST

```rust
pub fn process_bid<'info>(
    [...]
) -> Result<()> {
    [...]
    // Grab current expiry in case they're editing a bid
    let current_expiry = bid_state.expiry;
    // Figure out new expiry
    let expiry = match expire_in_sec {
        Some(expire_in_sec) => {
            let expire_in_i64 =
                i64::try_from(expire_in_sec).map_err(|_| TcompError::ExpiryTooLarge)?;
            require!(expire_in_i64 <= MAX_EXPIRY_SEC, TcompError::ExpiryTooLarge);
            Clock::get()?
                .unix_timestamp
                .checked_add(expire_in_i64)
                .ok_or(TcompError::ExpiryTooLarge)?
        }
        // When creating bid for the first time.
        None if current_expiry == 0 => Clock::get()?
            .unix_timestamp
            .checked_add(MAX_EXPIRY_SEC)
            .ok_or(TcompError::ExpiryTooLarge)?,
        // Editing a bid.
        None => current_expiry,
    };
    [...]
}
```

4. The `current_price` and `mm_fee` are redundantly calculated across several functions, including `calculate_fees`, `update_pool_accounting`, `pay_buyer_fees`, and `pay_seller_fees`. To improve efficiency, these values should be included in the `Fees` struct returned by `calculate_fees` and re-utilized in other functions to eliminate repetitive computations.
5. `assert_decode_mint_proof` is currently unutilized in the codebase and should be removed.

© 2024 Otter Audits LLC. All Rights Reserved.

Remediation

Remove the above-mentioned instances of unutilized or redundant code.

© 2024 Otter Audits LLC. All Rights Reserved.

Code Maturity OS-TFN-SUG-03

Description

1. In `marketplace`, `transfer_lamports_from_pda_min_balance` checks whether transferring the requested lamports will keep the recipient account rent-exempt after the transfer. If the transfer will leave the account below the rent-exempt threshold, the transfer is skipped entirely. However, this may result in some funds being stranded in the program-derived address if they are not enough to meet the threshold.
2. In the `marketplace`, there is a potential concern with mixing/matching instructions, where different types of NFTs may possibly be processed incorrectly.
3. `amm::close_expired_pool` should utilize `transfer_lamports_from_pda_min_balance` to safely transfer the remaining lamports from the PDA (Program Derived Address) of the pool account back to the original rent payer while ensuring the account balance never drops below the minimum required balance for account closure.

>_ *program/src/state/pool.rs* RUST

```rust
/// Closes a pool and returns the rent to the rent payer and any remaining SOL to the
    ↪  owner.
pub fn close_pool<'info>(
    pool: &Account<'info, Pool>,
    rent_payer: AccountInfo<'info>,
    owner: AccountInfo<'info>,
) -> Result<()> {
   [...]
    let pool_state_bond = Rent::get()?.minimum_balance(POOL_SIZE);
    let pool_lamports = pool.get_lamports();
    // Any SOL above the minimum rent/state bond goes to the owner.
    if pool_lamports > pool_state_bond {
        let owner_amount = unwrap_int!(pool_lamports.checked_sub(pool_state_bond));
        transfer_lamports_from_pda(&pool.to_account_info(), &owner, owner_amount)?;
    }
    // Rent goes back to the rent payer.
    pool.close(rent_payer)
}
```

4. Define `is_none` generically in `toolbox::nullable` for improved clarity.

© 2024 Otter Audits LLC. All Rights Reserved.

Remediation

1. Truncate the amount transferred, calculating the maximum possible amount to be transferred while keeping the recipient below the rent-exempt threshold.
2. Introduce a type discriminator for NFTs to ensure that each instruction is matched to the correct NFT type.
3. Utilize `transfer_lamports_from_pda_min_balance` in `close_expired_pool`.
4. Update `nullable` with a generic implementation of `is_none`.

Patch

1. Issue #1 resolved in b95be94.
2. Issue #3 resolved in 518f90b.

© 2024 Otter Audits LLC. All Rights Reserved.

Code Refactoring

OS-TFN-SUG-04

Description

1. In `wns::buy_spl`, `list_ta` should ideally be an associated token account (ATA) instead of an arbitrary token account as ATAs are linked directly to the owner and mint. This guarantees that the `list_ta` belongs to the `list_state` account, rendering it more secure because the association ensures that only the authorized owner may control the tokens.
2. It would be appropriate to round in favor of the pool by adjusting the rounding behavior in `pool::shift_price` during division operations to ensure that the outcome slightly benefits the pool rather than the user. This will prevent small losses to the pool due to repeated operations.
3. In `amm::pool`, the `POOL_SIZE` is incorrectly calculated as 452 instead of 447. `mm_fee_bps` is a `NullableOption<u16>` and will only occupy two bytes rather than the three bytes as originally allocated. Also, the `taker_sell_count` and `taker_buy_count` fields have been replaced by `price_offset`, which needs four bytes instead of the original eight bytes allocated for the two fields.

>_ program/src/state/pool.rs RUST

```rust
#[constant]
#[allow(clippy::identity_op)]
pub const POOL_SIZE: usize =
        8                               // discriminator
        + (2 * 1)                       // version + bump
        + 32                            // identifier
        + 8 * 3                         // created_at, updated_at, expiry
        + (2 * 1) + (2 * 8) + 1 + 3     // pool config
        + (3 * 32)                      // owner, whitelist, rent_payer
        + (32 + 8)                      // currency and currency amount
        + (3 * 4)                       // taker_sell_count, taker_buy_count, nfts_held
        + (2 * 4) + 8                   // pool stats
        + (3 * 32)                      // shared escrow, cosigner, maker_broker
        + 4                             // max_taker_sell_count
        + 100                           // _reserved
        ;
```

Remediation

Implement the above-mentioned refactors.

© 2024 Otter Audits LLC. All Rights Reserved.

Patch

1. Issue #1 resolved in `442a514`.
2. Issue #2 resolved in `17a69e2`.
3. Issue #3 resolved in `442a514`.

© 2024 Otter Audits LLC. All Rights Reserved.

Code Clarity

OS-TFN-SUG-05

Description

1. In the current implementation of the `marketplace`, if someone changes the owner of their Associated Token Account (ATA), it will abort. While there are no security implications, it does create a potential user experience issue. Users may list an item and later render it unavailable by changing the owner of their ATA.
2. Currently the code is manually calling exit to serialize account state.
3. `validate_mint` should be stricter, especially if the intention is to limit interactions with certain mint extensions or hook programs as implied in the comments, as currently only Libreplex is supported.
4. Utilize the `close` macro instead of `close_account` to simplify the process of closing accounts in the Anchor framework. The `close` macro automatically handles account closure at the end of the instruction, returning any rent-exempt balance to the specified destination.

>_ quorum_upgrade/sources/quorum_upgrade_policy.move RUST

```
public fun new(
    [...]
): QuorumUpgradeCap {
    // currently the allowed voters is limited to 100 and the number of
    // required votes must be at least 2 and less or equal than the number of voters
    assert!(vec_set::size(&voters) > 0, EAllowedVotersError);
    assert!(vec_set::size(&voters) <= 100, EAllowedVotersError);
    assert!(required_votes > 0, ERequiredVotesError);
    assert!(required_votes <= vec_set::size(&voters), ERequiredVotesError);
    [...]
}
```

5. Utilize `MintProof::PREFIX` in `whitelist::InitUpdateMintProof` for improved readability.

Remediation

1. During the listing process, lock the ATA's ownership so the user is restricted from changing it while the listing is active.
2. Utilize Anchor's default exit handler instead of manually exiting.
3. Implement stricter validation such that `validate_mint` aborts if any unexpected extensions or hook programs are encountered.
4. Utilize the `close` macro.
5. Incorporate the above suggestion.

© 2024 Otter Audits LLC. All Rights Reserved.

Code Optimizations OS-TFN-SUG-06

Description

1. Combine the `WithdrawMarginAccountCpiTAmm` and `WithdrawMarginAccountCpiTcomp` instructions to simplify the code, as their only differences are the seeds and discriminator checks.
2. In the compressed-NFT standard within `buy::process_buy`, replacing `validate_consigner` with `Option<Signer>` for the `consigner` account will improve efficiency.

>_ *marketplace/program/src/instructions/compressed/buy.rs* RUST

```
pub fn process_buy<'info>(
    [...]
) -> Result<()> {
    [...]
    let remaining_accounts =
        if let Some(remaining_account) = validate_cosigner(&ctx.accounts.cosigner,
            ↪  list_state)? {
            v.push(remaining_account);
            v.extend_from_slice(ctx.remaining_accounts);
            v.as_slice()
        } else {
            ctx.remaining_accounts
        };
    [...]
}
```

3. In `process_buy_t22_spl`, if the creator account is missing from the remaining accounts, it is more efficient to revert immediately rather than proceeding to fail later during the `transfer_creators_fee` step.

Remediation

Apply the above optimizations for increased efficiency.

© 2024 Otter Audits LLC. All Rights Reserved.

A — Vulnerability Rating Scale

We rated our findings according to the following scale. Vulnerabilities have immediate security implications. Informational findings may be found in the General Findings.

CRITICAL

Vulnerabilities that immediately result in a loss of user funds with minimal preconditions.

Examples:

- Misconfigured authority or access control validation.
- Improperly designed economic incentives leading to loss of funds.

HIGH

Vulnerabilities that may result in a loss of user funds but are potentially difficult to exploit.

Examples:

- Loss of funds requiring specific victim interactions.
- Exploitation involving high capital requirement with respect to payout.

MEDIUM

Vulnerabilities that may result in denial of service scenarios or degraded usability.

Examples:

- Computational limit exhaustion through malicious input.
- Forced exceptions in the normal user flow.

LOW

Low probability vulnerabilities, which are still exploitable but require extenuating circumstances or undue risk.

Examples:

- Oracle manipulation with large capital requirements and multiple transactions.

INFO

Best practices to mitigate future security risks. These are classified as general findings.

Examples:

- Explicit assertion of critical internal invariants.
- Improved input validation.

© 2024 Otter Audits LLC. All Rights Reserved.

B — Procedure

As part of our standard auditing procedure, we split our analysis into two main sections: design and implementation.

When auditing the design of a program, we aim to ensure that the overall economic architecture is sound in the context of an on-chain program. In other words, there is no way to steal funds or deny service, ignoring any chain-specific quirks. This usually requires a deep understanding of the program's internal interactions, potential game theory implications, and general on-chain execution primitives.

One example of a design vulnerability would be an on-chain oracle that could be manipulated by flash loans or large deposits. Such a design would generally be unsound regardless of which chain the oracle is deployed on.

On the other hand, auditing the program's implementation requires a deep understanding of the chain's execution model. While this varies from chain to chain, some common implementation vulnerabilities include reentrancy, account ownership issues, arithmetic overflows, and rounding bugs.

As a general rule of thumb, implementation vulnerabilities tend to be more "checklist" style. In contrast, design vulnerabilities require a strong understanding of the underlying system and the various interactions: both with the user and cross-program.

As we approach any new target, we strive to comprehensively understand the program first. In our audits, we always approach targets with a team of auditors. This allows us to share thoughts and collaborate, picking up on details that others may have missed.

While sometimes the line between design and implementation can be blurry, we hope this gives some insight into our auditing procedure and thought process.

© 2024 Otter Audits LLC. All Rights Reserved.